Technology Stack

Date	15 July 2026
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	2 Marks

Define Your Technology Stack

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the solution. A well-chosen technology stack ensures the application is scalable, maintainable, and aligned with the project requirements.

S. No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, custom CSS3, Vanilla JavaScript	No framework/build step needed for a small, focused UI; keeps the app lightweight, fast to load, and easy to deploy alongside Flask.
2	Backend / Server-Side	Python + Flask	Lightweight, minimal-boilerplate web framework that integrates directly with the Python ML stack (Pandas, scikit-learn, XGBoost) used for training and inference.
3	Database / Data Storage	Flat files: joblib model artifacts (.save) + CSV/Excel datasets	The app is a stateless single-prediction service with no user accounts or persistent records, so a traditional database is unnecessary; flat files keep deployment simple.
4	Cloud / Hosting / Deployment	Render (live at https://rising-waters-12fj.onrender.com/)	Free/simple deployment directly from GitHub with automatic builds, HTTPS by default, and a straightforward path to scale if traffic grows.
5	Version Control & CI/CD	GitHub (https://github.com/NandaGunasri/Rising-Waters) with Render auto-deploy on push	Enables version history, rollback, and continuous deployment — every push to the main branch triggers a fresh Render build without manual steps.
6	Third-Party APIs / Other Tools	None in production yet; NASA POWER / Copernicus APIs planned	Roadmap item to replace manually entered rainfall figures with live satellite/weather telemetry for real-time predictions.

Live deployment: https://rising-waters-12fj.onrender.com/ | **Source code:** https://github.com/NandaGunasri/Rising-Waters